

Итоговый проект. Разработка и развертывание ИИ-приложения в Kubernetes

Дисциплина: Оркестрация и контейнеризация

Проект (экзамен): Разработка и развёртывание ИИ-приложения в Kubernetes

Студент: Березняк В.Н., гр. М24-535

Среда выполнения: Ubuntu 22.04.5 LTS (VMware Workstation) + VS Code (Windows 11, Remote-SSH)

Дата: [вставить дату сдачи]

Этот файл — **методичка** для повторения шагов.

В местах с пометкой **СКРИНШОТ** вставляю **свои** скриншоты (файлы держу в папке `screenshots/`).

Структура репозитория (рекомендуемая)

```
.
├── Итоговый проект. Разработка и развертывание ИИ-приложения в Kubernetes.md
├── README.md
├── app/
│   ├── SentimentApplication.java
│   └── Dockerfile
├── k8s/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── hpa.yaml
│   └── service-monitor.yaml
└── screenshots/
    ├── 03_minikube/
    ├── 04_docker/
    ├── 05_k8s/
    ├── 06_monitoring/
    └── 07_trends/
```

1. Введение

1.1. Цель и задачи проекта

Цель проекта — продемонстрировать полный цикл разработки, контейнеризации и развертывания Java-приложения с простым анализом тональности текста в локальном кластере Kubernetes (Minikube), а также показать базовую интеграцию с системой мониторинга Prometheus + Grafana.

Задачи:

1. Разработать простое Java-приложение с REST API:
 - эндпоинт `GET /api/sentiment?text=...`, возвращающий JSON с оценкой тональности;
 - эндпоинт `GET /health` для проверки здоровья;
 - эндпоинт `GET /metrics` в формате Prometheus для сбора метрик.
2. Упаковать приложение в Docker-образ размером менее 150 МБ (multi-stage build).
3. Развернуть приложение в Kubernetes (Minikube) с:
 - Deployment минимум на 3 реплики;
 - Service типа `LoadBalancer`;
 - Ingress для маршрутизации запросов на `/api` и `/health`;
 - HPA (Horizontal Pod Autoscaler) по CPU.
4. Настроить мониторинг:
 - установка Prometheus + Grafana через Helm (`kube-prometheus-stack`);
 - подключение приложения через `ServiceMonitor`.
5. Подготовить раздел анализа тенденций по статьям arXiv (будет доработан отдельно).

1.2. Используемый технологический стек

Компонент	Технология
Язык/Runtime	Java SE 17
REST	<code>com.sun.net.httpserver.HttpServer</code>
Контейнеризация	Docker
Оркестрация	Kubernetes (Minikube)
Управление	<code>kubectl</code>
Мониторинг	Prometheus + Grafana (<code>kube-prometheus-stack</code>)
Балансировка	Service (LoadBalancer) + Ingress
Масштабирование	HPA по CPU

1.3. Среда выполнения

- Ubuntu 22.04.5 LTS (виртуальная машина в VMware Workstation)
- На хосте Windows 11: VS Code + расширения Remote Development / Remote-SSH
- Минимальные ресурсы VM (рекомендация): 4 vCPU, 8+ GB RAM, 20+ GB диск

2. Архитектура решения

2.1. Логическая схема

Компоненты решения:

1. **Sentiment API (Java-приложение)**
 - `/api/sentiment` — анализ тональности;
 - `/health` — health-проверка;

- `/metrics` — метрики в формате Prometheus.
- 2. **Kubernetes Deployment**
 - 3 реплики Pod'ов с контейнером `sentiment-app`;
 - probes (liveness/readiness), requests/limits.
- 3. **Service (LoadBalancer)**
 - единая точка входа и балансировка по репликам.
- 4. **Ingress**
 - маршрутизация HTTP на `/api` и `/health`.
- 5. **HPA**
 - автоскейлинг по CPU (3...10 реплик).
- 6. **Prometheus + Grafana**
 - сбор метрик Kubernetes и приложения через `ServiceMonitor`;
 - визуализация в Grafana.

(Схему/диаграмму добавлю позже — скрин или рисунок из редактора.)

3. Развёртывание инфраструктуры Minikube

3.1. Установка Minikube и kubectl

3.1.1. Установка Minikube

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-
amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

minikube version
```

СКРИНШОТ: `minikube version`

3.1.2. Установка kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

kubectl version --client
```

СКРИНШОТ: `kubectl version --client`

3.2. Создание кластера Minikube

Запуск кластера:

```
minikube start --cpus=4 --memory=8192mb --nodes=2
```

СКРИНШОТ: запуск `minikube start ...`

Проверка статуса:

```
minikube status  
kubectl get nodes
```

СКРИНШОТ: `minikube status, kubectl get nodes`

3.3. Включение необходимых аддонов

Для работы Ingress и HPA включаю аддоны `ingress` и `metrics-server`:

```
minikube addons enable ingress  
minikube addons enable metrics-server  
  
minikube addons list
```

СКРИНШОТ: `minikube addons list`

Для доступа к сервисам типа `LoadBalancer` использую:

```
minikube tunnel
```

`minikube tunnel` нужно держать запущенным в отдельном терминале.

СКРИНШОТ: запущенный `minikube tunnel`

4. Контейнеризация Java-приложения

4.1. Реализация REST-API анализа тональности

Приложение написано на Java SE 17 (без Spring), через `com.sun.net.httpserver.HttpServer`.

Эндпоинты:

- `GET /api/sentiment?text=...` → JSON с тональностью и score
- `GET /health` → `{"status":"UP"}`
- `GET /metrics` → метрика `sentiment_requests_total` (Prometheus text format)

Код приложения находится в файле:

- `app/SentimentApplication.java` (см. Приложение А)

4.2. Dockerfile и multi-stage build

Dockerfile с multi-stage сборкой и `jlink` для минимального JRE:

- `app/Dockerfile` (см. Приложение В)

4.3. Сборка и локальное тестирование образа

Сборка образа:

```
cd app
docker build -t sentiment-app:1.0 .
docker images sentiment-app:1.0
```

СКРИНШОТ: `docker images sentiment-app:1.0` (видно размер <150 MB)

Локальный запуск:

```
docker run --rm -p 8080:8080 --name sentiment-test sentiment-app:1.0
```

Проверка эндпоинтов (в другом терминале):

```
curl "http://localhost:8080/api/sentiment?text=I+love+Kubernetes"
curl "http://localhost:8080/health"
curl "http://localhost:8080/metrics"
```

СКРИНШОТ: curl-проверка `api/health/metrics`

4.4. Загрузка образа в Minikube

Вариант 1 (через `docker-env` Minikube):

```
eval "$(minikube docker-env)"
cd app
docker build -t sentiment-app:1.0 .
```

Вариант 2 (через `minikube image load`):

```
docker save sentiment-app:1.0 | minikube image load sentiment-app:1.0
```

СКРИНШОТ: загрузка образа в Minikube

5. Развёртывание приложения в Kubernetes

5.1. Kubernetes-манифесты

Файлы лежат в папке `k8s/`:

1. `k8s/deployment.yaml` — Deployment (3 реплики) + ServiceAccount.
2. `k8s/service.yaml` — Service `LoadBalancer`.
3. `k8s/ingress.yaml` — Ingress.
4. `k8s/hpa.yaml` — HPA по CPU.
5. `k8s/service-monitor.yaml` — ServiceMonitor для Prometheus.

5.2. Применение манифестов

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/ingress.yaml
kubectl apply -f k8s/hpa.yaml

kubectl get deployments
kubectl get pods -o wide
kubectl get svc
kubectl get ingress
kubectl get hpa
```

СКРИНШОТ: список ресурсов после `apply`

5.3. Тестирование доступа к приложению

5.3.1. Через port-forward

```
kubectl port-forward svc/sentiment-service 8080:80
curl "http://localhost:8080/api/sentiment?text=I+like+Kubernetes"
```

СКРИНШОТ: port-forward + curl

5.3.2. Через Ingress

1. Узнать IP Minikube:

```
minikube ip
```

2. Прописать host в `/etc/hosts` (на Ubuntu VM):

```
<MINIKUBE_IP> sentiment.local
```

3. Убедиться, что запущен `minikube tunnel` (см. п. 3.3)

Проверка:

```
curl "http://sentiment.local/api/sentiment?text=hello"
curl "http://sentiment.local/health"
```

СКРИНШОТ: curl по `sentiment.local`

5.4. Проверка работы HPA

Создаю нагрузку:

```
kubectl run load-generator --image=busybox --restart=Never -- /bin/sh -c 'while
true; do wget -q -O- http://sentiment-service/api/sentiment?text=load; done'
```

Слежение за масштабированием:

```
kubectl get hpa -w
kubectl get pods -w
```

СКРИНШОТ: рост нагрузки / изменение реплик

6. Мониторинг: Prometheus и Grafana

6.1. Установка Helm и kube-prometheus-stack

6.1.1. Установка Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
```

СКРИНШОТ: `helm version`

6.1.2. Установка стека мониторинга

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-
charts
helm repo update

kubectl create namespace monitoring

helm install prometheus prometheus-community/kube-prometheus-stack --namespace
monitoring --set
prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false --set
grafana.adminPassword=admin123 --wait
```

Проверка:

```
kubectl get pods -n monitoring
kubectl get svc -n monitoring
```

СКРИНШОТ: pods/svc в namespace monitoring

6.2. Подключение приложения к Prometheus (ServiceMonitor)

Применяю `ServiceMonitor`:

```
kubectl apply -f k8s/service-monitor.yaml
kubectl get servicemonitor -A
```

СКРИНШОТ: ServiceMonitor в кластере

6.3. Доступ к Prometheus и Grafana

```
kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090 &
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80 &
```

- Prometheus: <http://localhost:9090>
- Grafana: <http://localhost:3000> (логин `admin`, пароль `admin123`)

СКРИНШОТ: открытая Grafana / Prometheus

6.4. Настройка Grafana и дашборд

6.4.1. Data Source Prometheus

1. [Connections](#) → [Data sources](#) → [Add data source](#)
2. Prometheus
3. URL: <http://prometheus-kube-prometheus-prometheus:9090>
4. [Save & Test](#)

СКРИНШОТ: успешный [Save & Test](#)

6.4.2. Дашборд

Панели (минимум):

- [sentiment_requests_total](#) (по времени)
- CPU/Memory по pod'ам
- статус pod'ов

СКРИНШОТ: дашборд Grafana

7. Анализ тенденций (arxiv.org, 2024–2025 гг.)

Этот раздел буду **прорабатывать отдельно**. Здесь оставляю структуру и таблицу-шаблон.

7.1. Подход к выбору литературы

Критерии отбора статей:

- опубликованы в 2024–2025 гг.;
- связаны с AI-подходами в Kubernetes / контейнеризации / оркестрации;
- описывают scheduling / autoscaling / serverless-инференс / edge-архитектуры.

7.2. Таблица статей (шаблон, 5+)

№	Название	Авторы	Год	Ссылка arXiv	Тематика	Ключевые результаты
1	TODO	TODO	2024	TODO	TODO	TODO
2	TODO	TODO	2024	TODO	TODO	TODO
3	TODO	TODO	2024	TODO	TODO	TODO
4	TODO	TODO	2025	TODO	TODO	TODO
5	TODO	TODO	2025	TODO	TODO	TODO

СКРИНШОТЫ/артефакты (если нужны):

8. Заключение (заполню после выполнения)

8.1. Достигнутые результаты (шаблон)

- ☐ Реализовано Java-приложение `/api/sentiment, /health, /metrics`
- ☐ Собран Docker-образ <150MB
- ☐ Развёрнуто в Minikube: Deployment (3), Service LB, Ingress, HPA
- ☐ Установлен Prometheus+Grafana, собраны метрики, сделан дашборд
- ☐ Подготовлен анализ arXiv (будет позже)

8.2. Трудности и решения (таблица)

Трудность	Описание	Решение
TODO	TODO	TODO
TODO	TODO	TODO

8.3. Перспективы развития (шаблон)

- ☐ заменить rule-based на ML/LLM;
- ☐ добавить кэширование;
- ☐ CI/CD (GitHub Actions / GitLab CI);
- ☐ трейсинг (Jaeger/Tempo);
- ☐ service mesh (Istio/Linkerd).

9. Список использованных источников

9.1. Документация

- Minikube: <https://minikube.sigs.k8s.io/docs/start/>

2. Kubernetes Docs: <https://kubernetes.io/docs/home/>
3. Helm: <https://helm.sh/docs/>
4. kube-prometheus-stack: <https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack>
5. Grafana: <https://grafana.com/docs/>

9.2. Научные работы (будут добавлены позже)

- *TODO: список 3–5+ статей arXiv (2024–2025)*

Приложения (файлы проекта)

- **Приложение A:** `app/SentimentApplication.java`
- **Приложение B:** `app/Dockerfile`
- **Приложение C:** `k8s/*.yaml`